

PAPER_504: WOLFRAM_TERM Auto-Collection Framework

Unified Quantum Field Framework (UQFF)

Daniel T. Murphy • UQFF Research Consortium • 2026

Source: PAPER_504_WOLFRAM_TERM_AutoCollection_Framework.md

PAPER_504: WOLFRAM_TERM Auto-Collection Framework

Session: 137 | **Source:** grok_share_84a767d3.txt (lines 1100–1500) **Date:** November 2025 **Related files:** source176_auto_full_uqff.cpp, add_wolfram_terms.py

§1.1 Abstract

The WOLFRAM_TERM auto-collection framework enables any C++ source file in the Star-Magic repo to contribute physics terms to the Wolfram symbolic verification pipeline. Each file annotates its primary equation with a `#define WOLFRAM_TERM "..."` macro. At runtime, source176 scans all `.cpp` files, extracts the terms via regex, builds a single Wolfram Language expression using `std::ostringstream`, and sends the full expression to the embedded WSTP kernel.

§1.2 Injection Pattern

Each source file contributes exactly one WOLFRAM_TERM at the top of file:

```
// source42.cpp — SGR1745 Compressed MUGE
#define WOLFRAM_TERM &quot;g_SGR1745_compressed = (G*1.4e31/9.0e8^2)*(1 + 0.99*0.0005) + 1.114e-52*c^2/3&
```

The Python injector `add_wolfram_terms.py` automates insertion for all existing source files without one:

```
import os, re
for fname in os.listdir(&#39;&#39;):
    if fname.endswith(&#39;.cpp&#39;) and (fname.startswith(&#39;source&#39;) or fname == &#39;MAIN_1_CoA
        with open(fname, &#39;r+&#39;, encoding=&#39;utf-8&#39;) as f:
            content = f.read()
```

```

if &#39;WOLFRAM_TERM&#39; not in content:
    f.seek(0)
    f.write(f&#39;#define WOLFRAM_TERM &quot;+(*{fname}) 0&quot;\n&#39; + content)

```

§1.3 Auto-Collection Algorithm (source176)

Phase 1 – Locate repo root:

```

GetModuleFileNameW → exePath
root = exePath.parent_path().parent_path().parent_path()
// Climbs from: x64/Release/MAIN_1_CoAnQi.exe → x64/Release → x64 → repo root

```

Phase 2 – Scan files:

```

regex term_regex(R&quot;(#define\s+WOLFRAM_TERM\s*\&quot;(.*)\&quot;)&quot;);
for each .cpp in directory_iterator(root):
    read up to 500 lines
    if regex match found:
        terms.push_back(match[1])
    break // one term per file only

```

Phase 3 – Build expression ($O(N)$ via ostringstream):

```

ostringstream expr;
expr &lt;&lt; &quot;masterUQFF = &quot; &lt;&lt; terms[0];
for (i=1; i&lt;terms.size(); ++i)
    expr &lt;&lt; &quot; + &quot; &lt;&lt; terms[i];
expr &lt;&lt; &quot;;&quot;;

```

Phase 4 – Send to kernel:

```

count = terms.size()
megabytes = expr.str().size() / (1024*1024)
WolframEvalToString(expr.str())

```

Phase 5 – Simplify:

```

WolframEvalToString(&quot;$MaxExtraPrecision=10000; FullSimplify[masterUQFF]&quot;);

```

Phase 6 – Fallback:

```

if (terms.empty())
    WolframEvalToString(&quot;42^2&quot;); // alive test

```

§1.4 Performance Analysis: ostringstream vs String Concatenation

Method	Complexity	Behavior at N=827 terms
<code>std::string +=</code>	$O(N^2)$	Hours or crash before kernel receives anything
<code>std::ostringstream</code>	$O(N)$	Seconds, single allocation, sends in minutes

The ostringstream fix was the critical insight that moved the project from "stuck in file scan" to "kernel receiving the full expression."

§1.5 Progress Counter Output (progress prints)

```
Starting scan...
✓ source42.cpp → g_SGR1745_compressed = (G*1.4e31/9.0e8^2)*(1 + 0.99...
...
Scanned 10 files...
[Progress every 10 files]
Scan complete: 175 files, 827 terms found.
Built 827 terms (14.3 MB) – sending to kernel...
```

§1.6 WOLFRAM_TERM Catalog Statistics (Nov 2025)

Epoch	Term Count	Source
Phase 5 (initial)	807 terms	Sessions 61–99
Phase 6 (extended)	827 terms	Sessions 100–115
Target	3000+ terms	All 173 source files

§1.7 Equations

```
N_terms      = number of source files with WOLFRAM_TERM defined
scan_time    ≈ N_files × 500_lines / (disk_speed)      [seconds]
build_time   ≈ N_terms × mean_term_length / (mem_bandwidth) [seconds]
transfer_time ≈ expression_size_bytes / WSTP_bandwidth [seconds]
simplify_time ≈ f(N_terms, depth_of_cancellations)    [hours to days]
```

§1.8 Citation

Source: grok_share_84a767d3.txt, lines 1100–1500, 2700–3000 Related: source176_auto_full_uqff.cpp, add_wolfram_terms.py Commit: 8ae9ffe — "Fix WSTP integration: wolfram.exe launch mode + fix infinite scan loop" Paper number: PAPER_504